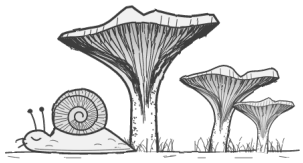
The Finite Element Method



Mikkel Paltorp (mpasc@dtu.dk), 11 May 2026
Department of Applied Mathematics and Computer Science, Technical University of Denmark

The aim of the Finite Element Method (FEM) is to solve (or more precisely approximate the solution of) a Partial Differential Equation (PDE) using a computer. In general the solution to a PDE is a *function*. Unfortunately it is computationally intractable to task a computer with finding this underlying function. The idea to resolve this issue is to parametrize a family of functions with the aim of computing the parameters through solving a linear system of equations. There is a plethora of ways to get to such a linear system of equations, with the most common being the so-called Galerkin approach.

Contents

1. Getting the computer to understand functions
2. The Finite Element Method for the Helmholtz equation
3. What is an element?
4. Computing the stiffness and mass matrices
5. Examples
Bibliography

1. Getting the computer to understand functions

In the most basic terms, computers are only capable of understanding numbers, which means that they are inherently unable to solve equations where the unknowns are functions. This is a problem when trying to solve differential equations. To solve this problem, the functions are instead approximated using parameterizations for which the coefficients (numbers) are unknown. Intuitively, these numbers are exactly what the computer is asked to find. For element methods, this parameterization is chosen to be the simplest possible: a linear combination of functions

$$p(\mathbf{x}) \approx \mathbf{T}(\mathbf{x})\mathbf{p} = \begin{bmatrix} T_1(\mathbf{x}) & T_2(\mathbf{x}) & \dots & T_{n(\mathbf{x})} \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}, \tag{1}$$

where p is the unknown function being approximated. Note that the linearity is with respect to the unknown parameters $\mathbf{p}$, but not necessarily in the known basis functions $\mathbf{T}(\mathbf{x})$.

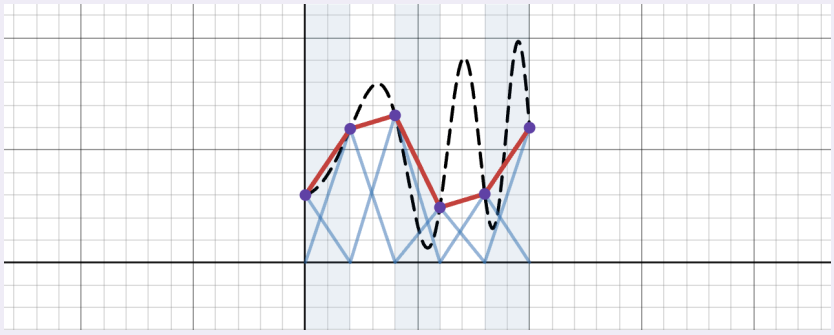
One might ask: *How does the above relate to the Finite Element Method?* The answer is that the functions T_i are chosen to be simpler functions with support equal to only a few subdomains of the original domain. These subdomains are commonly referred to as elements.

Interactive figure 1: Piecewise polynomial approximation

Interactive figure 1 (Piecewise polynomial approximation) In one dimension an approximation of the form Equation 1 is easy to visualize. The graph shows the function p (dashed) and the approximation $\mathbf{T}(x)\mathbf{p}$ (solid) on N elements (every other one shaded). The coefficients $p_i = p(x_i)$ are the large dots, and each basis function T_i, scaled by its p_i, is drawn faint. There are two ways to improve the approximation: more elements[1] or basis functions of a higher polynomial order p.[2]

[1] Called h-refinement, as it decreases the element size $h = 1/N$.

[2] Called p-refinement. Here p is the order, not the function being approximated.



Interactive version on the website →

2. The Finite Element Method for the Helmholtz equation

In acoustics we are often interested in solving the Helmholtz equation, given as

$$\Delta p(\mathbf{x}) + k^2 p(\mathbf{x}) = 0, \quad \mathbf{x} \in \Omega, \tag{2}$$

where k is most often referred to as the wavenumber. In its current form the basis functions $\mathbf{T}(\mathbf{x})$ must be twice differentiable; piecewise linear functions, for example, have a vanishing second derivative inside each element and cannot be used. This constraint can be relieved using the so-called weak formulation. In order to do so we multiply the Helmholtz equation with a so-called test function φ followed by an integration over the domain Ω

$$\int_\Omega \varphi(\mathbf{x})(\Delta p(\mathbf{x}) + k^2 p(\mathbf{x}))\,\mathrm{d}\mathbf{x} = 0. \tag{3}$$

Utilizing integration by parts on the term including the Laplacian we can move derivatives from $p(\mathbf{x})$ onto $\varphi(\mathbf{x})$ as

$$\int_\Omega (\nabla\varphi(\mathbf{x}))^\top \nabla p(\mathbf{x})\,\mathrm{d}\mathbf{x} - k^2 \int_\Omega \varphi(\mathbf{x}) p(\mathbf{x})\,\mathrm{d}\mathbf{x} - \int_{\partial\Omega} \varphi(\mathbf{x}) \frac{\partial p}{\partial n}(\mathbf{x})\,\mathrm{d}\mathbf{x} = 0. \tag{4}$$

In the above the constraint on the smoothness of $p(\mathbf{x})$ has been reduced to first order. Furthermore the last term

$$\int_{\partial\Omega} \varphi(\mathbf{x}) \frac{\partial p}{\partial n}(\mathbf{x})\,\mathrm{d}\mathbf{x} \tag{5}$$

can be used to include the boundary conditions into the equation. For simplification purposes we will in the following assume that the above is equal to zero. As such we have

$$0 = \int_\Omega (\nabla\varphi(\mathbf{x}))^\top \nabla p(\mathbf{x})\,\mathrm{d}\mathbf{x} - k^2 \int_\Omega \varphi(\mathbf{x}) p(\mathbf{x})\,\mathrm{d}\mathbf{x}. \tag{6}$$

We will now use the Galerkin approach to discretize the above equation. Note that "Galerkin" simply refers to the approach where $\varphi(\mathbf{x})$ and $p(\mathbf{x})$ are discretized using the same basis functions ($\mathbf{T}(\mathbf{x})$). In short this means that we introduce

$$p(\mathbf{x}) \approx \mathbf{T}(\mathbf{x})\mathbf{p}, \quad \varphi(\mathbf{x}) \approx \mathbf{a}^\top \mathbf{T}(\mathbf{x})^\top, \tag{7}$$

where $\mathbf{a} \in \mathbb{C}^n$ is a set of arbitrary coefficients that parametrize a whole *family* of test functions and $\mathbf{p} \in \mathbb{C}^n$ are the coefficients that we aim to find. Inserting the approximations it follows that

$$\begin{aligned} 0 &= \int_\Omega (\nabla \varphi(\mathbf{x}))^\top \nabla p(\mathbf{x}) \, \mathrm{d}\mathbf{x} - k^2 \int_\Omega \varphi(\mathbf{x}) p(\mathbf{x}) \, \mathrm{d}\mathbf{x} \\ &\approx \int_\Omega \mathbf{a}^\top (\nabla \mathbf{T}(\mathbf{x}))^\top \nabla \mathbf{T}(\mathbf{x}) \mathbf{p} \, \mathrm{d}\mathbf{x} - k^2 \int_\Omega \mathbf{a}^\top \mathbf{T}(\mathbf{x})^\top \mathbf{T}(\mathbf{x}) \mathbf{p} \, \mathrm{d}\mathbf{x} \\ &= \mathbf{a}^\top \left(\int_\Omega (\nabla \mathbf{T}(\mathbf{x}))^\top \nabla \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \right) \mathbf{p} - k^2 \mathbf{a}^\top \left(\int_\Omega \mathbf{T}(\mathbf{x})^\top \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \right) \mathbf{p} \\ &= \mathbf{a}^\top \left(\int_\Omega (\nabla \mathbf{T}(\mathbf{x}))^\top \nabla \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} - k^2 \int_\Omega \mathbf{T}(\mathbf{x})^\top \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \right) \mathbf{p}. \end{aligned} \tag{8}$$

We can make the above equality hold for all possible test functions, i.e. for all $\mathbf{a} \in \mathbb{C}^n$, if we find $\mathbf{p}$ such that

$$\left(\int_\Omega (\nabla \mathbf{T}(\mathbf{x}))^\top \nabla \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} - k^2 \int_\Omega \mathbf{T}(\mathbf{x})^\top \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \right) \mathbf{p} = \mathbf{0}. \tag{9}$$

This is exactly what we do in Finite Element computations. As a shorthand the above can be written as

$$(\mathbf{K} - k^2 \mathbf{M}) \mathbf{p} = \mathbf{0}, \tag{10}$$

where $\mathbf{K}$ is referred to as the *stiffness matrix* and $\mathbf{M}$ as the *mass matrix*. In the literature the equation can also be found as

$$(c^2 \mathbf{K} - \omega^2 \mathbf{M}) \mathbf{p} = \mathbf{0}, \tag{11}$$

where c is the propagation speed and ω is the angular frequency.

3. What is an element?

A key insight is that the element serves two purposes: it represents a subdomain of the original domain (also referred to as the geometry) while also describing parts of the unknown function(s) of interest. In general this subdomain is described by a parameterization, i.e. the element, as

$$\mathbf{x}^e(\mathbf{u}) = \mathbf{X}^e \mathbf{N}^e(\mathbf{u}) \in \Omega^e, \quad \forall \mathbf{u} \in \mathcal{L}^e, \tag{12}$$

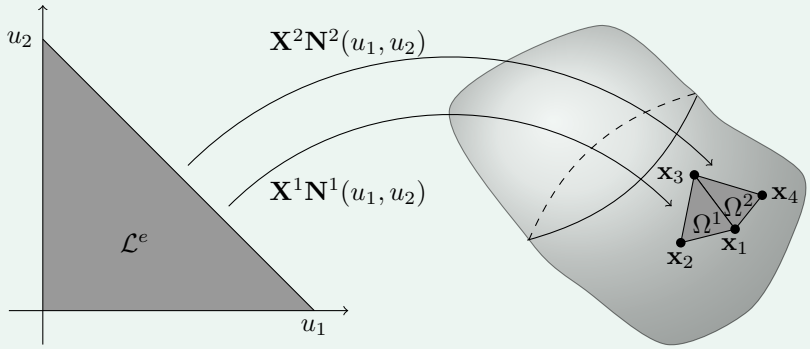
where the superscript e denotes the element number, $\mathbf{X}^e$ is a matrix with columns equal to the interpolation nodes of the geometry, $\mathbf{N}^e(\mathbf{u})$ are the so-called shape functions, Ω^e is the element in global coordinates and $\mathcal{L}^e$ are the local coordinates. In addition to the geometric interpolation of each element, we need to further define interpolations of the unknown functions, which in acoustics is usually taken as the pressure $p(\mathbf{x})$. On element e this interpolation can be done as

$$p(\mathbf{x}^e(\mathbf{u})) = \mathbf{T}(\mathbf{x}^e(\mathbf{u}))\mathbf{p} = \underbrace{\mathbf{T}(\mathbf{x}^e(\mathbf{u}))(\mathbf{L}^e)^\top}_{\mathbf{T}^e(\mathbf{u})} \underbrace{\mathbf{L}^e \mathbf{p}}_{\mathbf{p}^e} = \mathbf{T}^e(\mathbf{u}) \mathbf{p}^e, \quad \mathbf{u} \in \mathcal{L}^e, \tag{13}$$

where $\mathbf{L}^e$ is a permutation-like matrix that extracts the relevant values of $\mathbf{p}$ and the relevant functions of $\mathbf{T}(\mathbf{x})$ and orders them such that they correspond to the local values $\mathbf{p}^e$ and the local basis functions $\mathbf{T}^e(\mathbf{u})$.

Example 1: Linear triangular elements

Example 1 (Linear triangular elements)



The linear shape functions for a triangular element *can* have the form

$$\mathbf{N}^e(u_1,u_2) = \begin{bmatrix} 1-u_1-u_2 \\ u_1 \\ u_2 \end{bmatrix}, \quad u_1 \in [0,1], u_2 \in [0,1-u_1]. \tag{14}$$

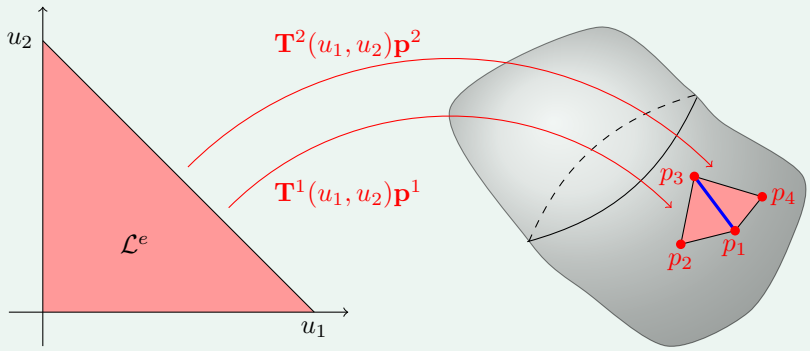
The choice in the wording is because the ordering of the columns of $\mathbf{X}^e$ can change the ordering of the rows of $\mathbf{N}^e(\mathbf{u})$ or vice versa. This is something that one should keep in mind in practice when using different mesh file formats. Taking the second element of the figure above as an example, it could be that

$$\mathbf{X}^2 = [\mathbf{x}_3 \ \ \mathbf{x}_1 \ \ \mathbf{x}_4]. \tag{15}$$

Note that extending the geometric interpolation to higher orders is as simple as adding more rows/functions to $\mathbf{N}^e(u_1,u_2)$ as well as more columns/points to $\mathbf{X}^e$.

Example 2: Basis functions for continuous linear interpolation

Example 2 (Basis functions for continuous linear interpolation)



Continuous linear basis functions on triangular elements are similar to shape functions for a linear triangular element and differ only in the fact that they are the transpose:

$$\mathbf{T}^e_{\text{continuous}}(u_1,u_2) = [1-u_1-u_2 \ \ u_1 \ \ u_2], \quad u_1 \in [0,1], u_2 \in [0,1-u_1], \tag{16}$$

where the subscript "continuous" is only there to highlight that it is a continuous formulation. Again, the ordering of the columns of the row vector depends on the ordering of the element corners.

Example 3: Element localization matrix

Example 3 (Element localization matrix) For a continuous linear element e all three corners correspond to a value of the global vector $\mathbf{p}$. For example, the second element in the continuous plot has local corner values given by $\mathbf{p}^2 = [p_3 \;\; p_1 \;\; p_4]^\top$. This element would have $\mathbf{L}^e$ given as

$$\mathbf{L}^2 = \begin{bmatrix} 0 & 0 & 1 & 0 & \dots & 0 \\ 1 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 1 & \dots & 0 \end{bmatrix}, \tag{17}$$

so that $\mathbf{p}^2 = \mathbf{L}^2\mathbf{p}$. Note that $\mathbf{L}^e$ is only an artifact of the mathematical description. Any reasonable implementation should use indexing instead of multiplication with $\mathbf{L}^e$.

4. Computing the stiffness and mass matrices

Using the element description of both the geometry and the pressure function the mass matrix can be computed as follows

$$\begin{aligned} \mathbf{M} &= \int_\Omega \mathbf{T}(\mathbf{x})^\top \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \\ &\approx \sum_{e=1}^{N} \int_{\Omega^e} \mathbf{L}_e^\top \mathbf{L}_e \mathbf{T}(\mathbf{x})^\top \mathbf{T}(\mathbf{x}) \mathbf{L}_e^\top \mathbf{L}_e \, \mathrm{d}\mathbf{x} \\ &= \sum_{e=1}^{N} \mathbf{L}_e^\top \left(\int_{\mathcal{L}^e} \mathbf{T}^e(\mathbf{u})^\top \mathbf{T}^e(\mathbf{u}) \; \text{jacobian}(\Omega^e, \mathbf{u}) \, \mathrm{d}\mathbf{u} \right) \mathbf{L}_e \\ &\approx \sum_{e=1}^{N} \mathbf{L}_e^\top \left(\underbrace{\sum_{i=1}^{Q} w_i \mathbf{T}^e(\mathbf{u}_i)^\top \mathbf{T}^e(\mathbf{u}_i) \; \text{jacobian}(\Omega^e, \mathbf{u}_i)}_{\mathbf{M}_e} \right) \mathbf{L}_e, \end{aligned} \tag{18}$$

where the first approximation is due to the geometry being approximated by the elements, and the second is due to the quadrature with the Q points $\mathbf{u}_i$ and weights w_i. Furthermore, $\text{jacobian}(\Omega^e, \mathbf{u})$ is a function that describes the distortion from the local coordinates $\mathcal{L}^e$ onto the global element Ω^e. This distortion represents length, area, or volume depending on the dimensionality of the problem. Note that if the elements are the same size then $\mathbf{M}_e$ is constant with respect to each element (this is the case in the code below).

The stiffness matrix is computed similarly but with a difference in the fact that the gradient must have its coordinates changed from global to local. Using the chain rule it follows that

$$\nabla_{\mathbf{u}} p(\mathbf{x}(\mathbf{u})) = \mathbf{J}(\mathbf{u}) \nabla_{\mathbf{x}} p(\mathbf{x}(\mathbf{u})), \tag{19}$$

where $\mathbf{J}(\mathbf{u}) = (\partial \mathbf{x} / \partial \mathbf{u})^\top$, i.e. $J_{ij} = \big(\partial x_j\big)/(\partial u_i)$, so that $\text{jacobian}(\Omega^e, \mathbf{u}) = |\det \mathbf{J}(\mathbf{u})|$. From this we can isolate the global gradient as

$$\nabla_{\mathbf{x}} p(\mathbf{x}(\mathbf{u})) = \mathbf{J}(\mathbf{u})^{-1} \nabla_{\mathbf{u}} p(\mathbf{x}(\mathbf{u})). \tag{20}$$

Putting things together the stiffness matrix can therefore be computed as

$$
\begin{aligned}
\mathbf{K} &= \int_{\Omega} (\nabla \mathbf{T}(\mathbf{x}))^{\top} \nabla \mathbf{T}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \\
&\approx \sum_{e=1}^{N} \int_{\Omega^e} \mathbf{L}_e^{\top} \mathbf{L}_e (\nabla_{\mathbf{x}} \mathbf{T}(\mathbf{x}))^{\top} \nabla_{\mathbf{x}} \mathbf{T}(\mathbf{x}) \mathbf{L}_e^{\top} \mathbf{L}_e \, \mathrm{d}\mathbf{x} \\
&= \sum_{e=1}^{N} \mathbf{L}_e^{\top} \left(\int_{\mathcal{L}^e} (\nabla_{\mathbf{x}} \mathbf{T}^e(\mathbf{u}))^{\top} \nabla_{\mathbf{x}} \mathbf{T}^e(\mathbf{u}) \ \mathrm{jacobian}(\Omega^e, \mathbf{u}) \, \mathrm{d}\mathbf{u} \right) \mathbf{L}_e \\
&= \sum_{e=1}^{N} \mathbf{L}_e^{\top} \left(\int_{\mathcal{L}^e} (\nabla_{\mathbf{u}} \mathbf{T}^e(\mathbf{u}))^{\top} (\mathbf{J}(\mathbf{u})^{\top})^{-1} \mathbf{J}(\mathbf{u})^{-1} \nabla_{\mathbf{u}} \mathbf{T}^e(\mathbf{u}) \ \mathrm{jacobian}(\Omega^e, \mathbf{u}) \, \mathrm{d}\mathbf{u} \right) \mathbf{L}_e \\
&\approx \sum_{e=1}^{N} \mathbf{L}_e^{\top} \left(\underbrace{\sum_{i=1}^{Q} w_i (\nabla_{\mathbf{u}} \mathbf{T}^e(\mathbf{u}_i))^{\top} \left(\mathbf{J}(\mathbf{u}_i)^{\top} \right)^{-1} \mathbf{J}(\mathbf{u}_i)^{-1} \nabla_{\mathbf{u}} \mathbf{T}^e(\mathbf{u}_i) \ \mathrm{jacobian}(\Omega^e, \mathbf{u}_i)}_{\mathbf{K}_e} \right) \mathbf{L}_e .
\end{aligned}
\tag{21}
$$

In the following code we implement three different ways of assembling the mass and stiffness matrices. Note that the first two implementations are only for educational purposes as they are both based on dense matrices.

Example 4: Matrix assembly implementations

Example 4 (Matrix assembly implementations)

julia

```julia
### Importing relevant packages
using FastGaussQuadrature
using BenchmarkTools
using LinearAlgebra
using SparseArrays
using ForwardDiff
using Random
Random.seed!(42)
## Defining geometry.
# Impedance tube of diameter 10 cm and length 1 m (used later)
D = 0.1             # 100 mm diameter
L = 10*D            # Length of the cavity
ne = 600            # Number of quadratic elements
nnt = 2*ne+1        # Total number of nodes
h = L/ne            # Length of the elements
x = Vector(0:h/2:L) # Coordinates table

## Computing the element matrices
# Local basis functions (gradient by ForwardDiff: slow but easy)
Tᵉ(u)  = [u .* (u .- 1)/2; 1 .- u .^2; u .* (u .+ 1)/2]'
∇Tᵉ(u) = hcat(ForwardDiff.derivative.(Tᵉ,u)...)
# All elements are equal, so the Jacobian is the same for each.
# It maps [-1,1] onto [x_i,x_{i+1}], i.e. length 2 onto length h.
jacobian(u) = h/2
# In 1D the Jacobian function and matrix are equal (not so in 2D).
J(u) = h/2
# The element matrices, the same for all elements of equal size.
Q = 3  # Number of Gauss points
u,w = gausslegendre(Q)
Me = sum(i -> w[i]*Tᵉ(u[i])'*Tᵉ(u[i])*jacobian(u[i]),1:Q)
Ke = sum(i -> w[i]*∇Tᵉ(u[i])'*J(u[i])^(-1)*J(u[i])^(-1)*∇Tᵉ(u[i])*
              jacobian(u[i]),1:Q)

## Assembly 1: with the localization matrices (never do this!)
function assembly1(Me, Ke, nnt, ne)
    K = zeros(nnt, nnt)  # Dense matrix! Not ideal!
    M = zeros(nnt, nnt)  # Dense matrix! Not ideal!
    for ie = 1:ne
        Le = zeros(3, nnt)
        Le[:, ie*2-1:ie*2+1] = Diagonal(ones(3))
        K += Le'*Ke*Le
        M += Le'*Me*Le
```

```
    end
    return K, M
end
@btime K, M = assembly1(Me, Ke, nnt, ne)

## Assembly 2: with indexing instead of localization matrices
function assembly2(Me, Ke, nnt, ne)
    K = zeros(nnt, nnt)  # Dense matrix! Not ideal!
    M = zeros(nnt, nnt)  # Dense matrix! Not ideal!
    for ie = 1:ne
        K[ie*2-1:ie*2+1, ie*2-1:ie*2+1] += Ke
        M[ie*2-1:ie*2+1, ie*2-1:ie*2+1] += Me
    end
    return K, M
end
@btime K, M = assembly2(Me, Ke, nnt, ne)

## Assembly 3: sparse, using the compact support of the elements
function assembly3(Me, Ke, nnt, ne)
    I = zeros(Int64, 4nnt - 3)
    J = zeros(Int64, 4nnt - 3)
    Kd = zeros(ComplexF64, length(I))
    Md = zeros(ComplexF64, length(I))
    for ie=1:ne
        for i = 1:3
            r = (8*(ie-1)+1 + 3*(i-1)):(8*(ie-1) + 3*i)
            I[r]  .= ie*2-1:ie*2-1+2
            J[r]  .= (ie-1)*2 + i
            Kd[r] += Ke[:, i]
            Md[r] += Me[:, i]
        end
    end
    K = sparse(I, J, Kd)
    M = sparse(I, J, Md)
    return K, M
end
@btime K, M = assembly3(Me, Ke, nnt, ne)
```

5. Examples

Example 5: Impedance tube

Example 5 (Impedance tube) The procedure used in an impedance tube to measure the surface impedance and absorption coefficient of an acoustic material works as follows. A loudspeaker is installed at one end ($x = L$) and an acoustic material of thickness h and surface impedance $\hat{Z}$ is bonded onto the other rigid and impervious end, with the surface of the material at $x = 0$. Below the cut-off frequency of the tube, only plane waves propagate and the problem is amenable to a one-dimensional analysis.

In this case the surface impedance of the material can be obtained from the measurement of the transfer function $\hat{H}_{12} = \hat{p}_2/\hat{p}_1$ between two microphones adequately placed in the tube (e.g., standard ASTM E1050):

$$\frac{\hat{Z}}{\rho_0 c_0} = \frac{1+R}{1-R}, \quad R = \frac{\hat{H}_{12} - \exp(-ik_0 s)}{\exp(ik_0 s) - \hat{H}_{12}} \exp(2ik_0(d+s)), \tag{22}$$

where d is the distance from microphone 2 to the sample, s is the spacing between the two microphones, and $k_0 = \omega/c_0$ is the wavenumber of air. The normal-incidence

absorption coefficient α is directly obtained from the reflection coefficient R: $\alpha = 1 - |R|^2$.

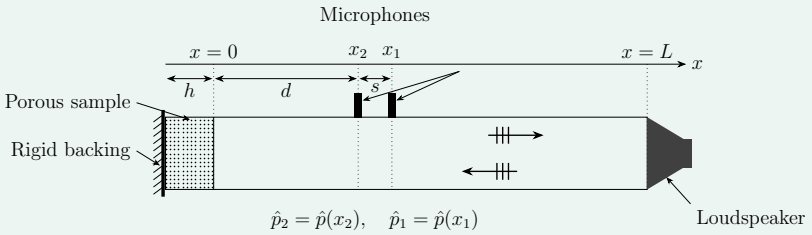


Figure 1: The impedance tube, with the surface of the material at $x = 0$ and the loudspeaker at $x = L$.

The impedance of a material of thickness h bonded onto a rigid wall (at $x = -h$) and excited by a plane wave with incidence angle $\theta = 0$ is given by [1]:

$$\hat{Z} = -i\hat{Z}_c \cot\left(\hat{k}_c h\right). \tag{23}$$

In the following the loudspeaker is modeled as a displacement of $U_L = 1$. Furthermore, the specific details of the setup are:

- Diameter of impedance tube $D = 10$ cm (and thus cut-off frequency $1.84c_0/(\pi D) \approx 2004$ Hz).
- Length of impedance tube: $L = 10D$.
- Microphone spacing: $s = D/2$.
- Distance between microphone 2 and the surface of the material: $d = D/2$.
- Thickness of the material: $h = 2$ cm.
- Characteristic impedance $\hat{Z}_c = \rho_0 c_0[1 + 0.057X^{-0.754} - i0.087X^{-0.732}]$ (where $X = \rho_0 f/\rho_p$).
- Wavenumber $\hat{k}_c = (\omega/c_0)[1 + 0.0978X^{-0.700} - i0.189X^{-0.595}]$ (where $X = \rho_0 f/\rho_p$).
- Density and propagation speed: $\rho_0 = 1.2$ kg/m³, $c_0 = 342.2$ m/s.
- Flow resistivity of the material: $\rho_p = 10000$ Rayls/m.

A 1D model of an impedance tube with a loudspeaker at $x = L$ and a material at $x = 0$ can be described by the following set of equations

$$\begin{cases} \frac{\mathrm{d}^2 p}{\mathrm{d}x^2}(x) + k_0^2 p(x) = 0, \quad x \in [0, L] \\ \frac{\mathrm{d}p}{\mathrm{d}x}(L) = -i\rho_0\omega v_L = \rho_0\omega^2 U_L \\ \frac{\mathrm{d}p}{\mathrm{d}x}(0) = ik_0\beta p(0) \end{cases} \tag{24}$$

where $\beta = \rho_0 c_0/\hat{Z}$ is the surface admittance of the material, normalized by the characteristic impedance of air, and U_L is the displacement of the loudspeaker, with velocity $v_L = i\omega U_L$. Here, and in the formulas above, time-harmonic quantities vary as $e^{i\omega t}$.

The PDE is modeled using the FEM as previously

$$(\mathbf{K} - k_0^2\mathbf{M})\mathbf{p} = \mathbf{0}. \tag{25}$$

However, in this case $\int_{\partial\Omega} \varphi(x) \frac{\partial p}{\partial n}(x)\,\mathrm{d}x \neq 0$. Instead we have

$$\begin{aligned}\left[\varphi(x)\frac{\mathrm{d}p}{\mathrm{d}x}(x)\right]_0^L &= \mathbf{a}^\top\mathbf{T}(L)^\top\frac{\mathrm{d}p}{\mathrm{d}x}(L) - \mathbf{a}^\top\mathbf{T}(0)^\top\frac{\mathrm{d}p}{\mathrm{d}x}(0)\\ &= \mathbf{a}^\top\left(\begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix}\rho_0\omega^2U_L - \begin{bmatrix}1\\0\\\vdots\\0\end{bmatrix}ik_0\beta p(0)\right)\\ &= \mathbf{a}^\top\left(\begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix}\rho_0\omega^2U_L - \mathrm{diag}\left(\begin{bmatrix}ik_0\beta\\0\\\vdots\\0\end{bmatrix}\right)\mathbf{p}\right).\end{aligned} \tag{26}$$

In the above we utilized the prescribed boundary conditions as well as the definition of $\mathbf{T}(x)$, so that $p(0) = p_1$ is the first entry of $\mathbf{p}$. Putting things together we find that

$$\left(\mathbf{K} - k_0^2\mathbf{M} + \mathrm{diag}\left(\begin{bmatrix}ik_0\beta\\0\\\vdots\\0\end{bmatrix}\right)\right)\mathbf{p} = \begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix}\rho_0\omega^2U_L. \tag{27}$$

Using the previously implemented FEM routines we can numerically solve the problem, and use the analytical expression to check the results, as shown in Figure 2.

julia

```julia
include("assembly_1d.jl")
using CairoMakie
## Recomputing FEM matrices
K,M = assembly3(Me,Ke,nnt,ne)
## Setup
ρ₀ = 1.2   # Fluid density
c₀ = 342.2 # Speed of sound
U_L = 1    # Piston displacement
D = 0.1    # Diameter of Impedance Tube
s = 0.5*D  # Microphone spacing
d = D/2    # Distance between mic 2 and sample
## Frequency domain
fc = floor(1.84*c₀/D/pi) # Cut off frequency
freq = Vector(100:2:fc)
ω = 2*pi*freq
k₀ = ω/c₀
## Impedance properties
Z₀ = ρ₀ * c₀
thickness = 0.02  # thickness of the material
σ = 10000 # flow resistivity
X = ρ₀*freq/σ
Zc = Z₀  *(1 .+ 0.0570*X.^(-0.754) - im*0.087.*X.^(-0.732))
kc = k₀ .*(1 .+ 0.0978*X.^(-0.700) - im*0.189.*X.^(-0.595))
Z = -im.*Zc.*cot.(kc*thickness) / Z₀
beta = 1.0 ./ Z   # convert to admittance
# Finding nodes located on the microphones
in2 = findall(x->x,abs.(x .- d) .< 1e-6)[1]
in1 = findall(x->x,abs.(x .- (d+s)) .< 1e-6)[1]
# Correcting the s and d distances to fit the numerical values
s = abs(x[in1] - x[in2])
d = x[in2]

## Output
ndof = nnt
nfreqs = length(ω)
P_mic1 = zeros(ComplexF64,nfreqs)
P_mic2 = zeros(ComplexF64,nfreqs)
A = Diagonal(zeros(ComplexF64,nnt))
F = zeros(ComplexF64,ndof)

## Frequency sweep
```

```
for i in eachindex(ω)
    A[1,1] = im*k₀[i]*beta[i]
    F[end,1] = ρ₀*ω[i]^2*U_L
    S = K - k₀[i]^2*M + A
    p = S\F
    P_mic2[i] = p[in2]
    P_mic1[i] = p[in1]
end

# Calculate the normalized impedance
H₁₂ = P_mic2 ./ P_mic1
R = (H₁₂ - exp.(-im*k₀*s))./(exp.(im*k₀*s) - H₁₂) .*
    exp.(im*2*k₀ * (d + s))
Z_num = (1 .+ R)./(1 .- R)
## Comparison with the exact solution
fig = Figure(size=(800,320))
ax_re = Axis(fig[1,1],xlabel="Frequency (Hz)",ylabel="Real part")
lines!(ax_re,freq,real.(Z),label="Analytical")
lines!(ax_re,freq,real.(Z_num),label="FEM",linestyle=:dash)
ax_im = Axis(fig[1,2],xlabel="Frequency (Hz)",ylabel="Imaginary part")
lines!(ax_im,freq,imag.(Z))
lines!(ax_im,freq,imag.(Z_num),linestyle=:dash)
axislegend(ax_re)
fig
```

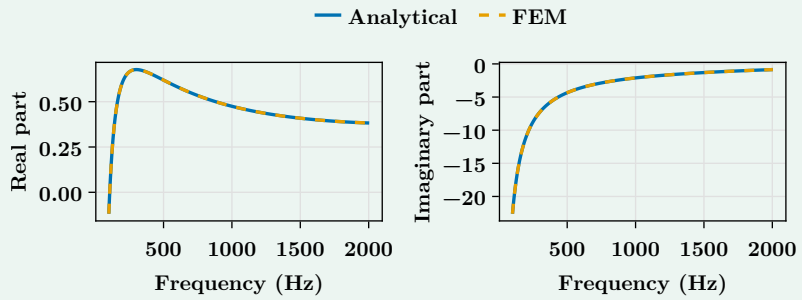


Figure 2: The normalized surface impedance $\hat{Z}/(\rho_0 c_0)$ of the material, analytical and computed with the FEM.

Example 6: Modes of a 10 × 4 m room

Example 6 (Modes of a 10 × 4 m room) The Finite Element Method can also be used to compute the modes, i.e. finding the eigenvalues ω^2 of the generalized eigenvalue problem

$$c^2\mathbf{K}\mathbf{p} = \omega^2\mathbf{M}\mathbf{p}. \tag{28}$$

For a rectangular room with side lengths L_x and L_y, here $L_x = 10$ m and $L_y = 4$ m, the frequencies of the modes can be computed analytically as

$$f_{n_x n_y} = \frac{c}{2}\sqrt{\left(\frac{n_x}{L_x}\right)^2 + \left(\frac{n_y}{L_y}\right)^2}. \tag{29}$$

The room is meshed with a regular grid of bilinear quadrilateral elements:

julia

```
using LinearAlgebra, CairoMakie
## Defining geometry.
Lx = 10
Ly = 4
n = 20+1
m = 8+1
ne = (n-1)*(m-1)    # Number of linear elements
```

```
nnt = n*m           # Total number of nodes
## Plotting the elements
x = Vector(0:Lx/(n-1):Lx)
y = Vector(0:Ly/(m-1):Ly)
X = kron(x,ones(m))
Y = kron(ones(n),y)
scatter(X,Y,axis=(aspect=DataAspect(),))
```

The Finite Element computation for the 2D room looks similar to the one for the 1D impedance tube. One difference is the reliance on the topology (connectivity) matrix, which lists the nodes of each element. The end of the script computes the analytical frequencies for comparison:

julia

```
include("modes_geometry.jl")
using ForwardDiff, SparseArrays, LinearAlgebra
using FastGaussQuadrature, Arpack
c₀ = 342.2
## Creating the topology
T = reshape(1:nnt,m,n)      # Numbering all nodes on the grid
topology = zeros(Int,4,ne)
topology[1,:] = T[1:end-1,1:end-1][:]
topology[2,:] = T[1:end-1,2:end][:]
topology[3,:] = T[2:end,2:end][:]
topology[4,:] = T[2:end,1:end-1][:]
## Computing Element matrices
# Local basis functions (gradient by ForwardDiff: slow but easy)
Tᵉ(u)  = [(1-u[1])*(1-u[2]); (1+u[1])*(1-u[2]);
          (1+u[1])*(1+u[2]); (1-u[1])*(1+u[2])]'/4
∇Tᵉ(u) = hcat(ForwardDiff.jacobian(Tᵉ,u))'
# All elements are equal, so the Jacobian is the same for each.
jacobian(u) = (x[2]-x[1])/2*(y[2]-y[1])/2
J(u) = Diagonal([x[2]-x[1];y[2]-y[1]])/2
# Defining the local element matrices.
Q = 2
u,wu = gausslegendre(Q)
v,wv = gausslegendre(Q)
U = kron(u,ones(Q))
V = kron(ones(Q),v)
W = kron(wu,wv)
P = [U';V']

Me = sum(i -> W[i]*jacobian(P[:,i])*Tᵉ(P[:,i])'*Tᵉ(P[:,i]),1:Q^2)
Ke = sum(i -> W[i]*jacobian(P[:,i])*∇Tᵉ(P[:,i])'*
              J(P[:,i])'^(-1)*J(P[:,i])^(-1)*∇Tᵉ(P[:,i]),1:Q^2)

## Assembly
function connected_topology(topology,nnt,ne)
    source_connections = [zeros(Int,0) for _ in 1:nnt]
    for element = 1:ne
        for i = 1:4
            node = topology[i,element]
            append!(source_connections[node],topology[:,element])
        end
    end
    sort!.(unique.(source_connections))
end
function create_I_J(connections)
    I = zeros(Int,sum(length.(connections)))
    J = zeros(Int,sum(length.(connections)))
    lower = 1
    for (idx,con) in enumerate(connections)
        upper = lower + length(con)
        I[lower:upper-1] .= con
        J[lower:upper-1] .= idx
        lower = upper
    end
```

```
    return I,J
end
function assembly(Me,Ke,topology,nnt)
    ne = size(topology,2)
    connections = connected_topology(topology,nnt,ne)
    I,J = create_I_J(connections)
    S = sparse(I,J,1:length(I))
    Kd = zeros(length(I))
    Md = zeros(length(I))
    for ie=1:ne
        top = topology[:,ie]
        Kd[S[top,top]] += Ke
        Md[S[top,top]] += Me
    end
    K = sparse(I,J,Kd)
    M = sparse(I,J,Md)
    return K,M
end
K,M = assembly(Me,Ke,topology,nnt);

using Arpack
F = eigs(c₀^2*K,M,which=:SM,nev=10);
freq = sort(sqrt.(abs.(F[1]))/2/π);

## The analytical frequencies, for comparison
nx, ny = 0:4, 0:4
freq_analytical = c₀/2*sqrt.((nx/Lx).^2 .+ (ny'/Ly).^2)
analytical_freqs = sort(freq_analytical[:])
println("FEM:        ", round.(freq[2:10],digits=2))
println("Analytical: ", round.(analytical_freqs[2:10],digits=2))
```

As seen in Figure 3, the FEM frequencies lie slightly above the analytical ones, and the difference grows with the frequency, as the mesh resolves the shorter wavelengths of the higher modes less accurately.

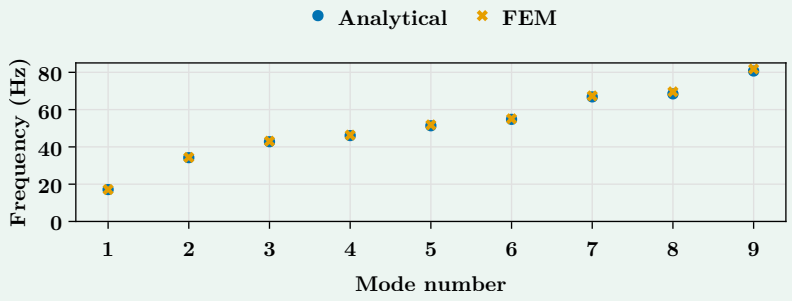


Figure 3: The first nine nonzero mode frequencies of the 10×4 m room, analytical and computed with the FEM.

Bibliography

[1] J.-F. Allard and N. Atalla, *Propagation of Sound in Porous Media: Modelling Sound Absorbing Materials*, 2nd ed. Wiley, 2009. doi: 10.1002/9780470747339.